

担当箇所の基本設計・詳細設計のたたき台

25G1053 齋藤翔太

2026 年 4 月 29 日

1 システムの基本設計：楽譜データ形式の基本方針

本システムでは、4 種類の楽器パート（主旋律、伴奏など）の楽譜データを PC 側ではなく、各 Arduino ノード（node_02～05）のメモリ上に保持させる方針とする。

Arduino の限られたメモリ容量を圧迫しないよう、楽譜データは絶対的な時間（ミリ秒）の配列ではなく、「音の高さ」と「音の長さ（拍）」のペアからなる相対的なデータとして定義し、プログラムメモリ上に定数として埋め込む設計とする。これにより、指揮棒の入力による動的なテンポ変更にも柔軟に対応可能とする。

2 システムの詳細設計：楽譜データおよび通信仕様

2.1 楽譜データの構造設計

1. 課題曲のパート振り分け

課題曲（カエルの歌）を以下の 4 パートに分割し、各 Arduino へ振り分ける。

- node_02: 主旋律（メロディ）パート
- node_03: 輪唱パート 1
- node_04: 輪唱パート 2
- node_05: ベース・リズム伴奏パート

2. データの記述形式

各音符は、以下の 2 つの要素を組み合わせた構造体（または 2 次元配列）として定義する。

- 音の高さ (**Pitch**): MIDI ノート番号（例: 真ん中のド=60）。休符は 0 や -1 などの特定の数値で定義する。
- 拍・リズム (**Duration**): 四分音符を基準（1.0）とした相対的な音価（例: 八分音符=0.5, 二分音符=2.0）。

3. データ構造の具体例（カエルの歌 冒頭）

主旋律（node_02）に持たせるデータ配列のイメージを以下に示す。

順番	音階 (Pitch)	長さ (Duration)	実際の音
1	60	1.0	ド（四分音符）
2	62	1.0	レ（四分音符）
3	64	1.0	ミ（四分音符）
4	65	1.0	ファ（四分音符）
5	64	1.0	ミ（四分音符）
6	62	1.0	レ（四分音符）
7	60	1.0	ド（四分音符）
8	-1	1.0	（四分休符）

4. 実装イメージ

定義したデータ形式を Arduino の多次元配列として記述した例を以下に示す。

```
int score[][2] = {  
    {60, 1.0}, // ド  
    {-1, 1.0}, // 四分休符（1 拍分、音を鳴らさずに待機）  
    {62, 1.0}  // レ  
};
```

このように、音階と音価をセットで管理することで、テンポの変更にも柔軟に対応可能な設計としている。

2.2 Arduino から PC への通信フロー

各 Arduino（node_02～05）から PC（Processing）へ送る「音データ」の通信仕様および処理の流れを以下の図に示す。

- **送るタイミング:** 各 Arduino が自身の持つ楽譜データと現在のテンポを計算し、リアルタイムに送信する。
- **送る内容:** 「どのノードからか（楽器 ID）」「どの高さの音か（Pitch）」「音の強さ（Velocity）」の 3 要素をカンマ区切りなどで送信する。
- **データの粒度:** 音符 1 つが発音されるごとの「Note ON（発音開始）」単位とする。音の長さ（Duration）の管理は、PC 側で行うか、Arduino から指定時間後に「Note OFF（発音停止）」信号を別途送る仕様とする。

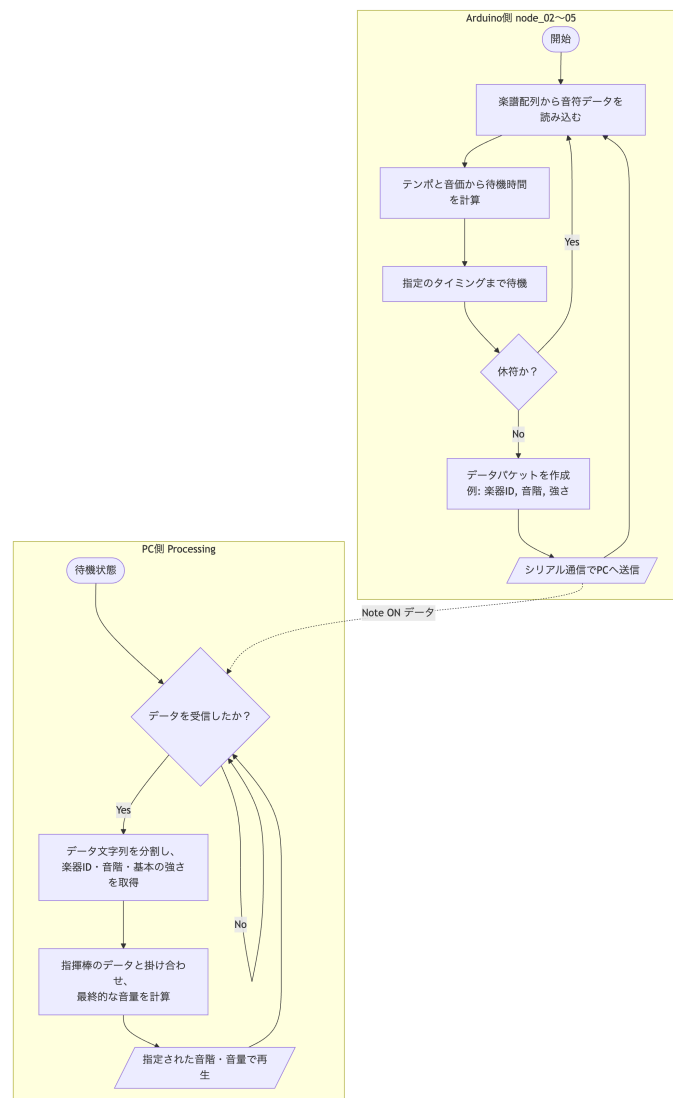


図1 Arduino と PC 間の通信および処理フロー

3 生成 AI の利用について

今回作成した基本設計・詳細設計のたたき台は、生成 AI の Gemini を利用し、文章の構成や、誤字脱字のチェックなどに活用した。